

“Eventify - Event Management & Ticketing Platform”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY**

By

Team ID: 3

G. BOOMIKA SRE 2520030010

P. NEHASREE 2520030174

P. HASINI 2520030282

Under the Esteemed Guidance of

Purushotham Sir

**Department of Computer Science
and Engineering**



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE
ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “Eventify - Event Management & Ticketing Platform” is a record of Bonafide work of G. BOOMIKA SRE - 2520030010 , P. NEHASREE - 2520030174, P. HASINI - 2520030282 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

G. BOOMIKA SRE – 2520030010

P. NEHASREE – 2520030174

P. HASINI – 2520030282

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY**



CERTIFICATE

This is certify that the mini project based report entitled“Eventify - Event Management & Ticketing Platform” is a bonafide work done and submitted by G. BOOMIKA SRE - 2520030010 , P. NEHASREE - 2520030174, P. HASINI - 2520030282

in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2026-2027.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Purushotham Sir, and my guide Purushotham Sir, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)/Dr. Kaja Shareef (CSIT)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

G. BOOMIKA SRE – 2520030010

P. NEHASREE – 2520030174

P. HASINI – 2520030282

ABSTRACT

Eventify is a web-based Event Management and Ticketing Platform developed to provide a single digital workflow for discovering events, registering users, booking tickets, generating digital tickets, and supporting ticket verification. The application separates the user interface, backend services, and persistent database so that each layer has a clear responsibility.

The frontend is implemented using React.js with Vite and provides Login, Signup, event listing, booking, and digital ticket display pages. The backend is implemented using FastAPI in Python and exposes REST endpoints for authentication, event retrieval, booking creation, QR-code generation, ticket verification, and check-in. MySQL stores the structured data required for users, events, bookings, tickets, and payment records.

A key feature of the system is multiple-ticket booking. When a user selects more than one ticket, the backend creates a separate unique ticket code for each ticket while retaining a single booking record with the requested quantity. QR images are generated through the Python qrcode library and displayed on the frontend. The system also stores a payment record and supports a checked-in status for ticket validation.

The project demonstrates practical full-stack integration, relational database design, authentication, REST API communication, QR-based digital ticketing, and browser-based testing. The current implementation is intended for local demonstration and can later be extended with a real payment gateway, organizer dashboard, richer event search, and production deployment.

INDEX

S.No.	Chapter	Topics
1	Introduction	Background, Problem Statement, Scope, Objectives, Importance, Target Users
2	System Requirements	Hardware, Software, Development Environment
3	Technology Stack	React.js, FastAPI, MySQL, Authentication, QR Generation
4	System Architecture	Architecture, Data Flow, Layer Description
5	Design	Database Schema and ER Diagram
6	Implementation	Authentication, Events, Booking, Payment Records, Tickets, QR, Check-in
7	Features	User and Booking Features
8	Testing	Frontend, Backend and Database Validation
9	Deployment	Local Setup and Execution
10	Challenges and Limitations	Development Issues and Current Limitations
11	Future Enhancements	Planned Extensions
12	Conclusion	Summary and Learning Outcomes
13	References	Technical Documentation
14	Appendix	Implementation Screenshots

CHAPTER 1 - INTRODUCTION

1.1 Background of the Project

Event management involves publishing event information, handling registrations, collecting bookings, issuing tickets, and validating attendance. When these activities are handled manually or across separate systems, users may need to repeatedly enter information and organizers may find it difficult to maintain consistent booking records. Eventify was developed as a centralized web application that connects these tasks through one interface.

The project combines a React.js frontend, a FastAPI backend, and a MySQL relational database. The frontend provides the visible user experience, while FastAPI handles validation, business logic, database communication, authentication, ticket generation, and verification. The database stores persistent records so that information remains available after the browser session ends.

1.2 Problem Statement

A simple event listing alone does not provide a complete ticketing workflow. The application must connect event selection with ticket quantity, booking information, payment records, and a unique digital ticket. Manual ticket issue and verification can also make it difficult to prevent duplicate use. Eventify addresses this requirement by providing a connected booking and digital-ticket process.

1.3 Scope

- User signup and secure login.
- Display of event name, date, venue, capacity, and ticket price.
- Booking of one or multiple tickets for a selected event.
- Storage of booking and payment records in MySQL.
- Unique ticket-code generation for each booked ticket.
- QR-code generation and display for digital tickets.
- Ticket verification and check-in support.

CHAPTER 1 - INTRODUCTION (CONTINUED)

1.4 Objectives

- Build a responsive React.js interface for event discovery and ticket booking.
- Develop FastAPI REST endpoints for authentication, event retrieval, booking, tickets, and verification.
- Store structured records in MySQL using related tables and foreign keys.
- Generate separate digital tickets when more than one ticket is booked.
- Provide QR-code based digital tickets for convenient presentation and verification.
- Maintain a clear separation between frontend, backend, and database responsibilities.

1.5 Importance of the Application

The application demonstrates how a traditional event-registration workflow can be converted into a structured digital process. Users can browse available events, select the required number of tickets, and receive digital ticket codes without depending on a manual ticket counter. For demonstration and academic purposes, the project also makes the database activity visible, allowing the booking record, payment record, ticket records, and check-in state to be inspected directly in MySQL Workbench.

1.6 Target Users

- Event attendees who want to browse and book tickets.
- Event organizers or administrators who need event and booking information.
- Project evaluators who need to verify the end-to-end frontend, backend, and database workflow.

CHAPTER 1 - INTRODUCTION (CONTINUED)

1.7 User Workflow

The normal user flow begins with account creation or login. After successful authentication, the user is redirected to the booking application, where upcoming events are retrieved from the FastAPI backend. The user selects an event, enters name and email information, chooses the number of tickets, and confirms the booking. The backend validates the event and quantity, calculates the total amount, stores the booking, creates the required ticket codes, and inserts a payment record. The frontend then displays the ticket codes and corresponding QR images.

For example, when two tickets are booked, the bookings table contains one booking row with tickets equal to 2. The tickets table contains two separate ticket rows because each ticket has its own unique code. This design makes it possible to display and verify tickets individually while retaining the original booking quantity.

1.8 Expected Outcome

The expected result is a functional web application in which a user can complete the journey from signup to digital ticket generation. The project also demonstrates database persistence, API integration, authentication, and QR generation in a single application.

CHAPTER 2 - SYSTEM REQUIREMENTS

2.1 Hardware Requirements

- Laptop or desktop computer with a modern multi-core processor.
- Minimum 8 GB RAM recommended for running VS Code, browser, MySQL Server and FastAPI together.
- At least 10 GB of free project storage, with additional space for packages and local database files.
- Stable internet connection for installing packages and accessing development resources.

2.2 Software Requirements

- Windows 10/11 or another operating system capable of running Node.js, Python and MySQL.
- Visual Studio Code for source-code development and debugging.
- Node.js and npm for the React/Vite frontend.
- Python with a virtual environment for the FastAPI backend.
- MySQL Server and MySQL Workbench for database storage and SQL operations.
- Google Chrome or another modern browser for application testing.

2.3 Functional Requirements

- Register and authenticate users.
- Retrieve events from the backend.
- Select an event and enter booking details.
- Validate ticket quantity against event capacity.
- Store booking and payment information.
- Generate ticket codes and QR images.
- Verify tickets and update check-in status.

CHAPTER 2 - SYSTEM REQUIREMENTS (CONTINUED)

2.4 Non-Functional Requirements

- Usability: the interface should provide clear navigation and readable booking forms.
- Reliability: invalid ticket quantities and unknown events should be rejected by the backend.
- Security: passwords are stored as bcrypt hashes and login uses JWT-based authentication.
- Maintainability: frontend, backend, and database responsibilities are separated.
- Performance: REST requests and database queries should complete quickly enough for interactive use.

2.5 Development Environment

During development the frontend is served by the Vite development server and the backend is served by Uvicorn. The frontend runs on localhost:5173 and the backend on localhost:8000. MySQL Server runs locally and provides the event_management database. This environment is sufficient for demonstration, testing, and project review.

2.6 Dependencies

- React and React Router for frontend components and navigation.
- FastAPI, Uvicorn and Pydantic for REST API development and validation.
- mysql-connector-python for MySQL connectivity.
- passlib/bcrypt for password hashing and verification.
- python-jose for JWT token generation.
- qrcode and its image support for QR generation.

CHAPTER 3 - TECHNOLOGY STACK

3.1 Frontend - React.js

React.js is used to build the interactive browser interface. The project uses reusable components/pages for Login, Signup, and Booking. React state is used for form fields, selected events, ticket quantity, ticket codes, booking messages, and the logged-in user. React Router provides navigation between authentication and booking pages.

3.2 Styling - HTML and CSS

The interface uses standard HTML elements and CSS for layout, spacing, gradients, cards, buttons, responsive grids, and authentication forms. CSS Grid is used to keep event cards side by side on larger screens, while media queries provide a single-column layout on smaller screens. Transitions are used to provide hover feedback.

3.3 Backend - FastAPI and Python

FastAPI is the REST service layer. It receives JSON requests from the React frontend, validates request data using Pydantic models, executes database operations, and returns JSON responses. Implemented routes include signup, login, event retrieval, booking creation, QR delivery, ticket verification, and check-in.

CHAPTER 3 - TECHNOLOGY STACK (CONTINUED)

3.4 Database - MySQL

MySQL is the relational database used for persistent storage. The implemented schema contains users, events, bookings, tickets, and payments. Foreign keys maintain the booking-to-event relationship and connect tickets and payments to bookings. Unique constraints help prevent duplicate user emails and ticket codes.

3.5 Authentication

During signup, the plain password is converted to a bcrypt hash before storage. During login, the provided password is verified against the stored hash. On successful login, the backend creates a JWT containing the user identity and role information. The frontend stores the token and user information in local storage for the current application session.

3.6 QR Generation

The backend uses the Python qrcode library to generate a QR image from each ticket code. The QR image is saved under the backend QR-code directory and is exposed to the frontend through the ticket QR endpoint. This allows each ticket to have an individual scannable representation.

3.7 Payment Record Handling

The current implementation records the booking amount and payment status in the payments table. The project does not depend on a live Razorpay gateway in the demonstrated version. A real external payment gateway can be integrated later while retaining the existing payment table and booking workflow.

CHAPTER 3 - TECHNOLOGY STACK (CONTINUED)

3.8 API Communication

The frontend communicates with the backend using HTTP requests and JSON payloads. CORS is configured in FastAPI for the local React origin so that browser requests are accepted during development. The communication pattern keeps UI logic separate from server-side validation and database operations.

3.9 Development and Testing Tools

- Visual Studio Code - coding and debugging.
- Vite - frontend development server.
- Uvicorn - ASGI server for FastAPI.
- MySQL Workbench - schema creation, queries and verification of stored data.
- Browser developer console - checking frontend requests and responses.

CHAPTER 4 - SYSTEM ARCHITECTURE

Eventify follows a simple three-layer application architecture. The browser provides the React.js frontend, the FastAPI service layer handles business rules and database communication, and MySQL stores persistent records. QR generation is handled by the backend using the Python qrcode library.

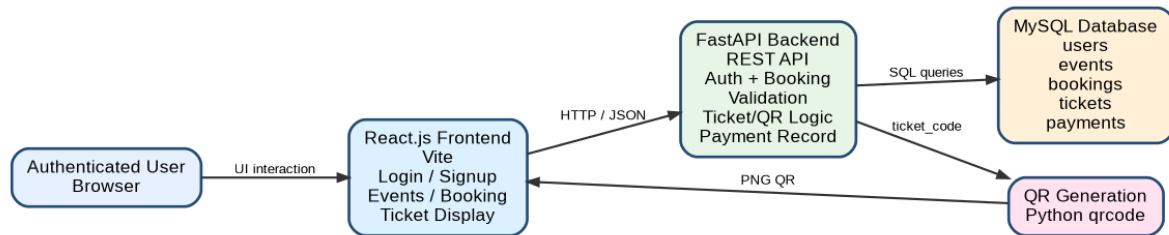


Figure 4.1 System Architecture of Eventify

4.1 Layer Description

- Presentation Layer: React.js pages, forms, navigation and ticket display.
- Application Layer: FastAPI routes, authentication, validation, booking logic, ticket creation and check-in.
- Data Layer: MySQL tables for users, events, bookings, tickets and payments.
- QR Service: Python qrcode library generates the digital ticket image from the unique ticket code.

CHAPTER 4 - SYSTEM ARCHITECTURE (CONTINUED)

4.2 Request and Response Flow

A typical event request begins when the React frontend sends GET /events to FastAPI. The backend executes a SELECT query on the events table and returns a JSON event list. During booking, the frontend sends a POST /bookings request containing event_id, customer_name, email and tickets. FastAPI validates the request, calculates the amount, creates the booking, generates ticket codes, inserts the payment record, and returns the resulting ticket codes and total amount.

4.3 Security and Validation Flow

Signup validates the submitted fields and stores a password hash. Login checks the supplied credentials and creates a signed JWT. Booking validation confirms that the selected event exists and that the ticket quantity is positive and does not exceed the event capacity. Ticket verification checks whether the supplied ticket code exists and whether it has already been checked in.

4.4 Local Deployment Architecture

For the current project demonstration, the frontend and backend run as two local processes. The frontend is started using npm run dev and the backend using uvicorn main:app --reload. MySQL Server runs locally and is accessed by the FastAPI process using the configured database connection.

CHAPTER 5 - DESIGN

5.1 Database Entities

The database is designed around five main tables. The users table stores account information; events stores event information; bookings stores customer booking requests; tickets stores individual digital tickets; and payments stores one payment record associated with a booking.

Table	Main Fields	Purpose
users	user_id, name, email, password_hash, role, created_at	User account
events	event_id, name, description, event_date, venue, capacity, ticket_price	Event catalogue
bookings	booking_id, event_id, customer_name, email, tickets, booking_date	Booking record
tickets	ticket_id, booking_id, ticket_code, checked_in, created_at	Individual ticket
payments	payment_id, booking_id, amount, payment_status, payment_date	Payment record

5.2 Relationship Design

One event can have many bookings, so events and bookings form a one-to-many relationship. One booking can generate multiple ticket rows when the user requests multiple tickets. A booking also has an associated payment record in the current implementation. User records are maintained independently in the users table; the current booking table stores customer name and email rather than a user_id foreign key.

CHAPTER 5 - DESIGN (CONTINUED)

5.3 ER Diagram

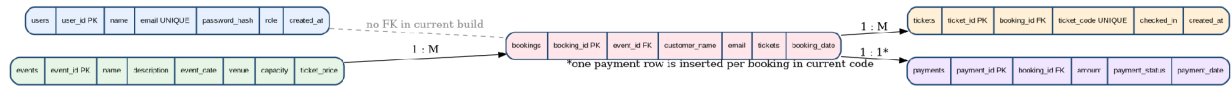


Figure 5.1 ER Diagram and Database Relationships of Eventify

5.4 Database Design Principles

- Primary keys uniquely identify records.
- Foreign keys connect bookings to events and connect tickets/payments to bookings.
- Unique constraints protect user email and ticket code uniqueness.
- Separate ticket rows support individual QR codes and check-in states.
- Payment information is separated from booking information for clear transaction records.

CHAPTER 6 - IMPLEMENTATION

6.1 Signup Module

The Signup page collects the user name, email and password. The React component sends these fields to POST /signup. FastAPI checks whether the email already exists, hashes the password using bcrypt through passlib, inserts the user record into the users table, and returns a success message. Duplicate email registration is rejected with an HTTP error response.

6.2 Login Module

The Login page accepts email and password. The POST /login route fetches the matching user record, verifies the password hash, generates a JWT, and returns user information. React stores the token and user details in local storage and navigates the authenticated user to the booking page.

6.3 Event Retrieval Module

The Booking page calls GET /events when it loads. FastAPI queries the events table and returns the available events. React maps the response into event cards showing the event date, venue, capacity, ticket price, and a Book Now button.

CHAPTER 6 - IMPLEMENTATION (CONTINUED)

6.4 Booking Module

When an event is selected, the booking form is displayed with the selected event name. The user enters customer information and ticket quantity. The total amount is calculated on the frontend for immediate feedback, while the backend independently calculates the amount using the event ticket price and requested quantity.

The booking endpoint first validates the event and ticket quantity. It then inserts one record into bookings. For each requested ticket, the backend generates a unique code using UUID logic and inserts an individual ticket row. A payment record is also inserted with the calculated amount and payment status. The transaction is committed only after the required records are prepared successfully.

6.5 Multiple Ticket Handling

If a customer books two tickets, the booking record stores tickets = 2, but the ticket table contains two unique ticket rows. This design allows each person or seat entry to receive a separate digital code while preserving one overall customer booking.

6.6 QR Ticket Display

The frontend stores the returned ticket codes in React state. For each code, it requests `/tickets/{ticket_code}/qr` and displays the returned QR image. The result is a ticket card containing the QR image and its unique ticket code.

CHAPTER 6 - IMPLEMENTATION (CONTINUED)

6.7 Ticket Verification and Check-In

The verification endpoint searches the tickets table using the unique ticket code. If the code does not exist, the ticket is invalid. If the ticket has already been checked in, the backend rejects reuse. Otherwise, the ticket is considered valid. The check-in endpoint then updates `checked_in` to `TRUE` after successful validation.

6.8 API Endpoints

Method	Endpoint	Purpose
POST	/signup	Create user account
POST	/login	Authenticate user and return JWT
GET	/events	Retrieve available events
POST	/bookings	Create booking and tickets
GET	/tickets/{ticket_code}/qr	Generate/display QR image
GET	/tickets/verify/{ticket_code}	Verify ticket
PUT	/tickets/checkin/{ticket_code}	Mark ticket as checked in

CHAPTER 7 - FEATURES

7.1 User Features

- Signup and Login with backend validation.
- Responsive navigation for Home, Events and Booking.
- Event cards with date, venue, capacity and ticket price.
- Selection of ticket quantity with capacity validation.
- Booking confirmation with calculated amount.
- Multiple digital tickets from one booking.
- Unique QR-based ticket display.

7.2 Booking Features

The booking process is designed around a small number of clear steps: choose an event, enter customer details, choose ticket quantity, confirm booking, and receive tickets. This reduces the amount of information the user must provide and keeps the final ticket details visible in the same application session.

7.3 Database Features

- Persistent storage of user accounts and event data.
- Separate booking and ticket records.
- Payment record associated with each booking.
- Unique ticket codes.
- Check-in state for individual tickets.

CHAPTER 7 - FEATURES (CONTINUED)

7.4 Validation Features

- Duplicate signup email is rejected.
- Invalid login credentials are rejected.
- Unknown event IDs are rejected during booking.
- Ticket quantity must be greater than zero.
- Requested ticket quantity cannot exceed event capacity.
- Unknown ticket codes are rejected during verification.
- Already checked-in tickets cannot be used again.

7.5 User Experience

The application uses a consistent blue-purple visual theme, responsive event cards, clear form spacing, rounded buttons, hover transitions, and a dedicated QR ticket section. The authentication pages use centered cards so that signup and login remain visually separate from the booking interface.

CHAPTER 8 - TESTING

8.1 Testing Approach

Testing was performed by running the frontend and backend locally, exercising the application through the browser, and checking the resulting records in MySQL Workbench. This approach validates the integration between the React interface, FastAPI endpoints, and database operations.

Test	Input	Expected Result	Result
Signup	Valid new user	User created	Pass
Signup	Existing email	Request rejected	Pass
Login	Valid credentials	Login and navigation succeed	Pass
Login	Invalid credentials	Error returned	Pass
Events	Open booking page	Events loaded from MySQL	Pass
Booking	1 ticket	1 booking + 1 ticket	Pass
Booking	2 tickets	1 booking + 2 ticket rows	Pass
Booking	Exceeds capacity	Booking rejected	Pass

CHAPTER 8 - TESTING (CONTINUED)

Test	Input	Expected Result	Result
Payment record	Successful booking	Payment row inserted	Pass
QR generation	Valid ticket code	QR image displayed	Pass
Verification	Valid ticket	Ticket marked valid	Pass
Verification	Invalid code	Ticket rejected	Pass
Check-in	Unused ticket	checked_in becomes TRUE	Pass
Check-in	Already used ticket	Reuse rejected	Pass

8.2 Database Verification Queries

- SELECT * FROM users; verifies account records.
- SELECT * FROM events; verifies event data.
- SELECT * FROM bookings; verifies booking quantity and customer information.
- SELECT * FROM payments; verifies stored payment records.
- SELECT * FROM tickets; verifies ticket code and check-in state.

8.3 Testing Result

The demonstrated workflows completed successfully during local testing. Database records matched the actions taken through the browser, including the important distinction that one booking can produce multiple ticket rows. QR images were displayed from the backend route, and ticket check-in state could be verified in MySQL.

CHAPTER 9 - DEPLOYMENT

9.1 Local Deployment Steps

- Start MySQL Server and ensure the event_management database exists.
- Open the backend terminal, activate the Python virtual environment, and run `uvicorn main:app --reload`.
- Open a second terminal in the React project and run `npm run dev`.
- Open the Vite URL in a browser and verify the application flow.

9.2 Environment Configuration

The backend uses local database connection settings and a local secret key for JWT signing during development. CORS allows requests from the React development origin. For a real deployment, secrets and database credentials should be moved into environment variables and a production ASGI process should be configured.

9.3 Current Deployment Status

The implemented project is demonstrated in a local development environment rather than a public cloud deployment. This is sufficient for the current academic review. Production deployment, HTTPS configuration, cloud database hosting, and a managed payment gateway are future extensions.

CHAPTER 10 - CHALLENGES AND LIMITATIONS

10.1 Challenges Faced

- Maintaining correct CORS communication between Vite and FastAPI.
- Keeping frontend, backend, and database field names consistent.
- Generating one unique ticket code per requested ticket.
- Displaying multiple QR tickets cleanly after one booking.
- Validating ticket quantities and preventing invalid bookings.
- Keeping authentication and page navigation synchronized.

10.2 Solutions Applied

The frontend API base URL was kept consistent, CORS was explicitly configured in FastAPI, and Pydantic request models were used for structured validation. The backend creates ticket records inside a loop based on the requested quantity, which ensures that every ticket receives a separate UUID-derived code. React stores the returned codes as an array so each QR image can be rendered independently.

CHAPTER 10 - CHALLENGES AND LIMITATIONS (CONTINUED)

10.3 Current Limitations

- The project is demonstrated locally and is not yet hosted publicly.
- Payment handling currently stores a payment record rather than integrating a live external gateway.
- The current UI focuses on attendee booking; a complete organizer dashboard is not part of the demonstrated build.
- The events are stored in MySQL rather than being synchronized with external event providers.
- No email/SMS notification service is currently connected.

10.4 Security Considerations

Passwords are not stored in plain text; bcrypt hashing is used. JWT is used for login sessions. For a production release, database credentials, JWT secrets, and other sensitive values should not be hard-coded in the source, and the application should use HTTPS, stronger secret management, and additional authorization controls.

CHAPTER 11 - FUTURE ENHANCEMENTS

- Integrate a live payment gateway such as Razorpay or Stripe.
- Add an organizer/admin dashboard for event creation, editing and booking analytics.
- Add email confirmation with ticket attachments or QR links.
- Add search and filtering by event date, venue and price.
- Provide downloadable ticket PDFs.
- Add event capacity tracking based on completed bookings.
- Introduce automated reporting and booking summaries.
- Deploy frontend, backend and database to secure cloud infrastructure.
- Add stronger role-based authorization and audit logs.

11.1 Future Payment Gateway

The current payment table provides a place to store amount and payment status. A future gateway integration could connect the booking flow to a real transaction provider while updating the existing payment record after confirmation. The booking should be finalized only after the payment response is validated.

11.2 Organizer Features

A future organizer dashboard can allow authorized users to create events, update event capacity, view bookings, inspect payment summaries, and verify tickets from a centralized interface. These features can be added without changing the basic React-FastAPI-MySQL architecture.

CHAPTER 12 - CONCLUSION

Eventify demonstrates a complete web-based event booking workflow using React.js, FastAPI, Python, MySQL, JWT/bcrypt authentication, and QR-code generation. The application connects user registration and login with event discovery, booking, payment record storage, digital ticket generation, and ticket verification.

The project successfully demonstrates full-stack communication. React handles the interface and state, FastAPI handles validation and business operations, and MySQL provides persistent relational storage. The multiple-ticket design is particularly important because it preserves one booking record while creating separate ticket records and QR codes for each requested ticket.

The development process provided practical experience in API design, database relationships, authentication, frontend-backend integration, validation, debugging, and testing with real database output. The current system is suitable for academic demonstration and provides a structured foundation for future enhancements such as real payment integration, organizer dashboards, notifications, and cloud deployment.

12.1 Skills Learned

- React component and state management.
- FastAPI REST API development.
- MySQL schema design and SQL verification.
- Password hashing and JWT authentication.
- QR-code based digital ticket generation.
- Frontend-backend integration and debugging.
- End-to-end application testing.

CHAPTER 13 - REFERENCES

- React.js Documentation - <https://react.dev/>
- FastAPI Documentation - <https://fastapi.tiangolo.com/>
- Python Documentation - <https://docs.python.org/3/>
- MySQL Documentation - <https://dev.mysql.com/doc/>
- Uvicorn Documentation - <https://www.uvicorn.org/>
- Python qrcode Library Documentation - <https://pypi.org/project/qrcode/>

Development and testing were also supported through Visual Studio Code, MySQL Workbench, the Chrome browser, and local package documentation used during implementation.

CHAPTER 14 - APPENDIX

14.1 Implementation Screenshots

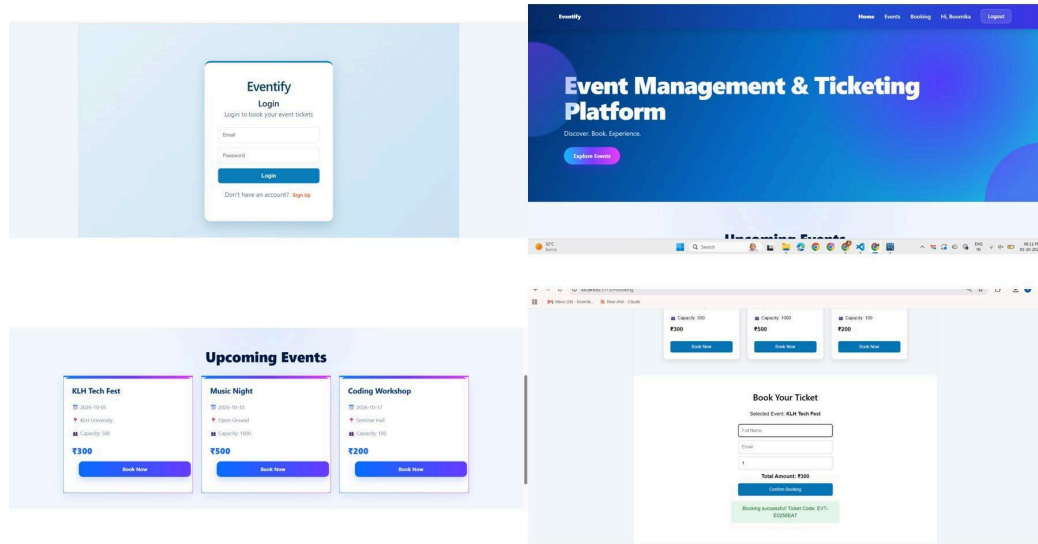


Figure 14.1 Selected Eventify Implementation Screenshots

CHAPTER 14 - APPENDIX (CONTINUED)

14.2 Demonstration Flow

- 1. Open Eventify and create an account.
- 2. Login using the registered credentials.
- 3. Browse the events retrieved from MySQL through FastAPI.
- 4. Select an event and book one or multiple tickets.
- 5. Verify the booking and payment records in MySQL Workbench.
- 6. Display the generated ticket codes and QR images.
- 7. Verify/check in a ticket and confirm the checked_in state in the database.

14.3 Core Database Demonstration Commands

```
USE event_management;  
SHOW TABLES;  
SELECT * FROM events;  
SELECT user_id, name, email, role FROM users;  
SELECT * FROM bookings ORDER BY booking_id DESC;  
SELECT * FROM payments ORDER BY payment_id DESC;  
SELECT * FROM tickets ORDER BY ticket_id DESC;
```